

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA5115 COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO5: *Develop the network security strategies based on the study of viruses, worms, firewall architectures, and IDS/IPS functionalities.CO (BL5, BL6)*

Assessment Tool Weightage: 10%

QUESTIONS: 20

TOTAL MARKS: 20

Annexure A MOCK INTERVIEW

Code of Conduct:

I M Nishanth (192511154) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M Nishanth

Annexure A

MOCK INTERVIEW

1. Analyze the weaknesses of SSL and explain why TLS is considered more secure in modern web systems.
2. Apply the TLS handshake process to explain how secure session keys are established between a client and server.
3. Compare SSL and TLS in terms of encryption mechanisms and identify key improvements in TLS.
4. Evaluate a scenario where improper certificate validation in TLS leads to a security breach. What could go wrong?
5. Analyze the role of digital certificates in SSL/TLS and explain how they ensure authentication.
6. Apply your understanding of SET protocol to explain how it protects credit card transactions in e-commerce.
7. Evaluate the limitations of the SET protocol in current online payment systems.
8. Compare SET protocol with TLS-based payment security and justify which is more practical today.
9. Analyze a malware attack scenario and differentiate whether it is caused by a virus, worm, or Trojan.
10. Apply appropriate mitigation techniques for a worm attack spreading across a corporate network.
11. Evaluate the impact of Trojan attacks on system confidentiality and integrity with an example scenario.
12. Compare viruses, worms, and Trojans based on propagation method and system impact.
13. Analyze the working of a packet-filtering firewall and identify its limitations in modern networks.
14. Apply firewall rules to block unauthorized access while allowing legitimate traffic in a given scenario.
15. Evaluate the effectiveness of stateful inspection firewalls over stateless firewalls.
16. Compare different types of firewalls and recommend the most suitable one for an enterprise network.

17. Analyze how an IDS detects malicious activities using signature-based and anomaly-based techniques.
18. Apply IDS/IPS mechanisms to prevent a distributed denial-of-service (DDoS) attack.
19. Evaluate the differences between IDS and IPS in terms of response to detected threats.
20. Propose improvements to reduce false positives in an IDS while maintaining high detection accuracy.

SIMATS ENGINEERING

ASSESSMENT TOOL 3

Mock Interview — Questions and Answers

1. Analyze the weaknesses of SSL and explain why TLS is considered more secure in modern web systems.

Answer: SSL (Secure Sockets Layer), especially SSLv2 and SSLv3, suffers from several well-documented weaknesses. It uses weak cryptographic primitives such as MD5 and SHA-1 for integrity checks, which are vulnerable to collision attacks. SSLv3 is susceptible to the POODLE attack, which exploits its padding scheme in CBC-mode ciphers to allow attackers to decrypt parts of encrypted traffic through a padding oracle. SSL also lacks robust protection against downgrade attacks, allowing a man-in-the-middle to force a connection to negotiate a weaker, older protocol version. Its handshake does not adequately protect the integrity of the negotiation itself, and its key derivation functions are cryptographically weaker than those used later.

TLS (Transport Layer Security) was designed to correct these flaws. TLS 1.2 and 1.3 use stronger hash functions (SHA-256 and above), authenticated encryption modes like AES-GCM and ChaCha20-Poly1305 that protect both confidentiality and integrity together, and improved key derivation (HKDF in TLS 1.3). TLS 1.3 removes support for outdated ciphers entirely, encrypts more of the handshake to reduce metadata leakage, and shortens the handshake to one round trip, which also reduces the attack surface. Because of these structural and cryptographic improvements, TLS is considered significantly more secure and is now the standard for HTTPS and other secure web communications, with SSL formally deprecated by the IETF.

2. Apply the TLS handshake process to explain how secure session keys are established between a client and server.

Answer: The TLS handshake establishes a shared secret session key between client and server without ever transmitting the key itself in the clear. In TLS 1.2, the process begins with a ClientHello, where the client proposes supported cipher suites, TLS versions, and a random nonce. The server responds with a ServerHello selecting the cipher suite and version, followed by its digital certificate (containing its public key) and, for ephemeral key exchange, its Diffie-Hellman or ECDHE parameters. The client verifies the certificate against a trusted Certificate Authority, generates its own key exchange parameters, and both sides independently compute the same pre-master secret using the exchanged parameters.

This pre-master secret, combined with the two random nonces, is fed into a key derivation function to produce the master secret, from which the actual session keys for encryption and MAC/authentication are derived. Both sides then send a ChangeCipherSpec and a Finished message, encrypted with the newly derived keys, to confirm that the handshake completed correctly and was not tampered with. From this point on, all application data is encrypted using the symmetric session keys. TLS 1.3 streamlines this into effectively one round trip by combining key shares into the initial hello messages, reducing latency while preserving forward secrecy through ephemeral Diffie-Hellman.

3. Compare SSL and TLS in terms of encryption mechanisms and identify key improvements in TLS.

Answer: SSL primarily relied on RC4, DES, and 3DES for bulk encryption, and MD5/SHA-1 for message authentication codes (MACs). These algorithms are now considered cryptographically weak: RC4 has statistical biases that can be exploited, and MD5/SHA-1 are vulnerable to collision attacks. SSL's MAC-then-encrypt construction in CBC mode also opened the door to padding oracle attacks such as POODLE. Key exchange in SSL commonly used static RSA, which does not provide forward secrecy — if the server's private key is ever compromised, all past recorded sessions can be decrypted retroactively.

TLS improved on every one of these fronts. Modern TLS versions use AEAD (Authenticated Encryption with Associated Data) ciphers like AES-GCM and ChaCha20-Poly1305, which combine encryption and integrity verification in a single, more secure step. TLS 1.2 introduced configurable, stronger hash functions (SHA-256 and above) for the handshake and MAC computations. TLS 1.3 mandates ephemeral Diffie-Hellman (ECDHE) key exchange, guaranteeing forward secrecy for every session, and drops legacy weak ciphers (RC4, DES, static RSA key exchange, CBC-mode ciphers with MAC-then-encrypt) entirely. Overall, TLS's improvements center on stronger algorithms, forward secrecy, integrated authenticated encryption, and a reduced, better-audited protocol surface.

4. Evaluate a scenario where improper certificate validation in TLS leads to a security breach. What could go wrong?

Answer: Consider a mobile banking application that implements a custom TLS client but fails to properly validate the server's certificate chain — for example, it accepts any certificate regardless of whether it is signed by a trusted Certificate Authority, or it does not check that the certificate's Common Name/Subject Alternative Name matches the domain being contacted. In this scenario, an

attacker positioned on the same network (such as a public Wi-Fi hotspot) can perform a man-in-the-middle attack: they intercept the connection, present their own self-signed or fraudulently obtained certificate, and the vulnerable app accepts it without complaint.

Because the app trusts the attacker's certificate, it unknowingly establishes a TLS session with the attacker instead of the real bank server. The attacker can then decrypt, read, and even modify all traffic — capturing login credentials, session tokens, and transaction details, and potentially injecting fraudulent transaction requests. This illustrates why certificate validation (chain of trust verification, expiry checking, hostname/SAN matching, and checking revocation status via CRL/OCSP) is not optional: it is the mechanism that binds the cryptographic channel to the identity the user actually intends to communicate with. Skipping or weakening this check reduces TLS's strong encryption to a false sense of security.

5. Analyze the role of digital certificates in SSL/TLS and explain how they ensure authentication.

Answer: Digital certificates bind a public key to an identity (typically a domain name or organization) and are issued and digitally signed by a trusted Certificate Authority (CA) after the CA verifies the requester's ownership of that identity. In SSL/TLS, the server presents its certificate during the handshake, and the client's TLS stack validates it by checking: (1) that the certificate is signed by a CA present in the client's trusted root store, following the chain of trust up to a root CA; (2) that the certificate has not expired or been revoked (via CRL or OCSP); and (3) that the domain name in the certificate matches the domain the client is trying to reach.

This process ensures authentication because only the legitimate holder of the private key corresponding to the certificate's public key can successfully complete the key exchange and prove possession of that key during the handshake (for example, by correctly decrypting or signing handshake data). An attacker without the private key cannot impersonate the server even if they somehow obtain a copy of the certificate, since certificates only contain the public key. This is what prevents man-in-the-middle attacks under normal conditions — certificates provide the trusted binding between identity and cryptographic key that the rest of the TLS session security depends on.

6. Apply your understanding of SET protocol to explain how it protects credit card transactions in e-commerce.

Answer: The Secure Electronic Transaction (SET) protocol was designed specifically to secure card-based payments over the internet by ensuring confidentiality, integrity, authentication, and non-repudiation among the three parties involved: the cardholder, the merchant, and the payment gateway/issuing bank. Its central innovation is Dual Signature, which cryptographically links order information (sent to the merchant) with payment information (sent to the bank) without either party seeing the other's data in full. The cardholder hashes both pieces of information together and signs the combined hash, so the merchant can verify the order matches the payment without ever seeing the card number, and the bank can verify the payment without seeing the order details.

In practice, when a customer places an order, SET encrypts the payment information using the bank's public key so only the bank can decrypt it — the merchant simply forwards this encrypted payload without being able to read the actual card details. Digital certificates issued to cardholders, merchants, and banks authenticate every party involved, preventing impersonation. This architecture protects credit card transactions by ensuring the merchant never has direct access to raw card numbers (reducing fraud and merchant liability), while the bank can independently authorize the payment, and both order and payment data are cryptographically bound so they cannot be tampered with or mismatched.

7. Evaluate the limitations of the SET protocol in current online payment systems.

Answer: Despite its strong security design, SET has largely fallen out of use in modern e-commerce due to several practical limitations. First, it required every participant — cardholder, merchant, and bank — to obtain and manage digital certificates, creating a heavy public key infrastructure (PKI) burden and significant deployment friction, especially for ordinary consumers who found certificate installation confusing. Second, SET required special client-side software (a 'digital wallet') to be installed on the buyer's machine, which limited adoption since most users were unwilling to install extra software just to shop online.

Third, SET transactions were computationally and procedurally heavier than simpler alternatives, since dual signatures and multiple certificate verifications added processing overhead and latency compared to just running a payment over a plain TLS-secured page. Fourth, the ecosystem needed near-universal adoption by banks, merchants, and payment processors to be effective, and this critical mass never materialized — competing simpler technologies won out. As a result, SET was effectively superseded by simpler, TLS-based payment security combined with tokenization and card-network protocols such as 3-D Secure, which achieve much of the same protection with far

less friction for end users and merchants.

8. Compare SET protocol with TLS-based payment security and justify which is more practical today.

Answer: SET was purpose-built for payment transactions, offering strong end-to-end guarantees: the merchant never sees the raw card number, dual signatures cryptographically bind order and payment data, and all parties are mutually authenticated via certificates. TLS-based payment security, by contrast, simply encrypts the communication channel between the browser and the merchant's server (and separately between the merchant and the payment gateway) — the merchant's server typically does see the card data in transit unless additional measures like tokenization are used, and there is no built-in dual-signature mechanism linking order and payment details.

In terms of practicality, TLS-based payment security wins decisively today. It requires no special client software — any standard browser supports it — and it leverages the existing, already-deployed HTTPS/PKI infrastructure. Combined with complementary technologies such as PCI-DSS compliance requirements, tokenization (replacing card numbers with non-sensitive tokens), and 3-D Secure (which adds cardholder authentication similar in spirit to SET's goals), TLS-based approaches deliver comparable real-world protection with dramatically lower deployment cost and complexity. This is precisely why the industry converged on TLS plus these complementary protocols rather than SET: strong theoretical security that nobody can practically deploy is less valuable than good security that is universally deployable.

9. Analyze a malware attack scenario and differentiate whether it is caused by a virus, worm, or Trojan.

Answer: Consider three related incidents in a company network. In the first, an employee opens an infected spreadsheet attachment; macro code embedded in the file attaches itself to other documents on the same machine and only spreads further when that infected document is manually shared with another user via email. This is a virus — it requires a host file and user action (opening/executing the host) to propagate, and it cannot spread on its own across the network.

In the second incident, a single unpatched machine on the network is found to be automatically scanning and infecting every other reachable machine with an exploitable vulnerability, with no user interaction at all, rapidly consuming bandwidth network-wide. This is a worm — it self-propagates autonomously across a network by exploiting vulnerabilities, without needing a host file or human action.

In the third incident, an employee downloads what looks like a legitimate PDF reader utility from an untrusted website; once installed, it silently opens a backdoor that lets an external attacker access the machine remotely. This is a Trojan — it disguises itself as legitimate, useful software to trick the user into installing it, and it does not self-replicate or attach to other files; instead it focuses on granting unauthorized access or performing a hidden malicious function.

10. Apply appropriate mitigation techniques for a worm attack spreading across a corporate network.

Answer: Containing an active worm outbreak requires both immediate and longer-term measures. Immediately, the affected network segment should be isolated — using VLAN segmentation or disabling switch ports — to stop the worm from reaching unaffected hosts, and internet/external connectivity for the segment should be temporarily cut if the worm is using it for command-and-control or further propagation. Network traffic should be monitored (via IDS/IPS and flow analysis) to identify the exploited vulnerability and the specific ports/protocols being abused, and firewall rules should be updated to block that traffic network-wide.

In parallel, the vulnerability the worm is exploiting must be identified and patched on all hosts, since the worm will continue re-infecting patched-but-still-vulnerable machines otherwise; this often means an emergency patch deployment cycle. Infected machines should be isolated, cleaned using updated antivirus/anti-malware signatures, and verified clean before being reconnected. For longer-term prevention, the organization should enforce timely patch management, network segmentation (so no single compromised host can reach the entire network), least-privilege access controls, egress filtering to limit outbound connections, and deploy an IDS/IPS tuned to detect the scanning and exploitation behavior characteristic of worm propagation.

11. Evaluate the impact of Trojan attacks on system confidentiality and integrity with an example scenario.

Answer: Trojans primarily compromise confidentiality and integrity by granting attackers covert access while appearing harmless. Consider a scenario where a finance department employee downloads a 'free invoice template tool' from an unofficial site. The tool functions normally but also installs a Remote Access Trojan (RAT) in the background. The attacker now has confidentiality impact: they can silently exfiltrate sensitive financial records, client data, and credentials stored on the machine, and can log keystrokes to capture banking portal logins — all without the user's knowledge.

The integrity impact follows from the same access: the attacker can modify financial records, alter

invoices or payment details (e.g., redirecting a wire transfer to their own account), install additional malware, or tamper with system logs to cover their tracks. Because the Trojan operates with the same privileges as the user who ran it, if that user has elevated access, the attacker inherits that access too, potentially compromising entire systems or databases. This example shows why Trojans are particularly dangerous compared to more 'visible' malware: they can persist undetected for long periods, silently undermining both the secrecy of data (confidentiality) and its trustworthiness/accuracy (integrity) simultaneously.

12. Compare viruses, worms, and Trojans based on propagation method and system impact.

Answer: Viruses require a host program or file to attach to and depend on human action — such as opening an infected file or running an infected program — to execute and spread. Their propagation is typically slower since it relies on users sharing infected files (via email, USB drives, downloads). Their system impact ranges from corrupting or deleting files to degrading performance, and since they need a host, cleaning usually means removing the infected file or restoring it.

Worms are self-replicating and self-propagating: they do not need a host file or human action, spreading autonomously across networks by exploiting vulnerabilities or misconfigurations. This makes them capable of extremely rapid, large-scale spread — potentially infecting thousands of machines within hours. Their impact is often felt at the network level first (bandwidth exhaustion, network congestion, denial of service) in addition to whatever payload they carry on individual hosts.

Trojans do not self-replicate or spread on their own at all; they rely entirely on social engineering, disguising themselves as legitimate or desirable software to trick a user into voluntarily installing them. Their propagation is therefore driven by deception rather than technical self-spreading. Their impact is typically the most targeted and severe on the individual system they infect — backdoors, data theft, credential harvesting, or remote control — because they are often built for a specific malicious purpose rather than indiscriminate spread.

13. Analyze the working of a packet-filtering firewall and identify its limitations in modern networks.

Answer: A packet-filtering firewall operates at the network layer (and sometimes transport layer) of the OSI model, examining each packet's header fields — source and destination IP address, source and destination port, and protocol type — against a predefined set of rules (an access control list). Based on this examination, it makes an independent allow-or-deny decision for every packet, without retaining any memory of past packets or the state of a connection. This makes it fast and

lightweight, suitable as a first line of defense at network boundaries.

However, this statelessness is also its core limitation: because each packet is evaluated in isolation, a packet-filtering firewall cannot verify whether a packet is part of a legitimate, already-established connection or determine the true context of the traffic. This makes it vulnerable to IP spoofing (an attacker forges a source address to match an allowed rule), and to attacks that craft packets to look superficially legitimate at the header level despite malicious payloads, since packet filters do not inspect payload content at all. It also cannot handle protocols that dynamically negotiate ports (like FTP) gracefully, and offers no application-layer awareness, making it blind to attacks embedded within otherwise 'allowed' traffic. Modern networks therefore layer packet filtering with stateful inspection and application-layer firewalls/proxies for adequate protection.

14. Apply firewall rules to block unauthorized access while allowing legitimate traffic in a given scenario.

Answer: Consider a corporate network where an internal web server (10.0.0.10) must be reachable from the internet only on HTTPS (port 443), an internal database server (10.0.0.20) must only be reachable from the web server and never directly from the internet, and all other inbound traffic from the internet should be denied by default. An appropriate rule set, applied in order and ending in an implicit/explicit deny-all, would be: (1) Allow inbound TCP traffic from any source to 10.0.0.10 on port 443. (2) Deny inbound traffic from any source to 10.0.0.10 on port 80/22/3389 and all other ports, closing off unnecessary management or legacy ports. (3) Allow inbound TCP traffic from 10.0.0.10 only to 10.0.0.20 on the database port (e.g., 3306 for MySQL). (4) Deny inbound traffic from any other source, including the general internet, to 10.0.0.20 entirely. (5) Deny all other inbound traffic by default (implicit deny).

For outbound traffic, rules should allow the web server to respond to established connections and reach the database server, while restricting the database server from initiating unsolicited outbound connections to the internet (to limit data exfiltration if it is ever compromised). This layered rule design follows the principle of least privilege: each system is granted only the minimum access it needs to perform its function, unauthorized paths (like direct internet-to-database access) are explicitly closed, and legitimate traffic paths (internet-to-web-server on 443, web-server-to-database) remain open and functional.

15. Evaluate the effectiveness of stateful inspection firewalls over stateless firewalls.

Answer: Stateful inspection firewalls maintain a connection state table that tracks the status of active connections (such as TCP handshake state, sequence numbers, and session context), rather than evaluating each packet in isolation as stateless (packet-filtering) firewalls do. This allows a stateful firewall to make far more intelligent decisions: it can permit inbound response packets only if they correspond to a connection that was legitimately initiated from inside the network, and reject packets that claim to be part of a connection but don't match any known session — effectively blocking many spoofing and unsolicited-connection attempts that a stateless firewall would allow through if the header fields happened to match an open rule.

This makes stateful firewalls significantly more effective at preventing attacks like session hijacking attempts, certain forms of IP spoofing, and unauthorized inbound connections disguised as replies, while also simplifying rule management since administrators typically only need to allow outbound-initiated traffic and the firewall automatically permits the corresponding legitimate replies. The trade-off is that stateful inspection requires more processing power and memory to maintain the state table, and can be susceptible to state-table exhaustion attacks (a form of denial of service) if not properly rate-limited. Even so, for most modern enterprise networks the security benefits of context-aware filtering far outweigh the additional resource cost, which is why stateful inspection is the baseline standard in nearly all commercial firewalls today.

16. Compare different types of firewalls and recommend the most suitable one for an enterprise network.

Answer: Packet-filtering firewalls operate at the network layer, checking header information only; they are fast and simple but offer no context-awareness or payload inspection. Stateful inspection firewalls add connection-tracking, offering much better protection against spoofing and unsolicited traffic while remaining reasonably fast, but still lack deep visibility into application-layer content. Application-layer (proxy) firewalls operate at Layer 7, inspecting the actual content of traffic (e.g., HTTP requests, email content), allowing them to block application-specific attacks like SQL injection or malicious file uploads, though at the cost of higher latency and processing overhead since they must fully parse each protocol. Next-Generation Firewalls (NGFWs) combine stateful inspection with deep packet inspection, intrusion prevention, application awareness, and often integrated threat intelligence, offering the broadest protection but at the highest cost and complexity.

For a typical enterprise network handling sensitive data, sizable traffic volumes, and facing sophisticated threats, a Next-Generation Firewall is the most suitable recommendation, ideally

deployed at the network perimeter alongside internal stateful/segmentation firewalls between critical zones (e.g., separating the DMZ, internal LAN, and database tier). This layered, defense-in-depth approach combines the speed of stateful filtering for internal segmentation with the deep, application-aware inspection of an NGFW at the perimeter, giving the enterprise both performance and comprehensive protection against modern, multi-vector attacks — something no single simpler firewall type can provide on its own.

17. Analyze how an IDS detects malicious activities using signature-based and anomaly-based techniques.

Answer: Signature-based (also called misuse-based) IDS detection works by maintaining a database of known attack patterns, or 'signatures' — specific byte sequences, packet structures, or behavioral patterns that have been previously identified as malicious (for example, the exact payload pattern of a known exploit or a specific string used by a particular malware family). Incoming traffic or system activity is compared against this signature database, and a match triggers an alert. This approach is highly accurate and produces very few false positives for known threats, since it is essentially pattern-matching against confirmed malicious indicators, but it is fundamentally unable to detect novel or zero-day attacks for which no signature yet exists, and it requires constant signature database updates to remain effective.

Anomaly-based detection instead builds a baseline or statistical model of what 'normal' network or system behavior looks like — typical traffic volumes, common protocols used, usual login times, typical process behavior — often using machine learning or statistical thresholds. Any activity that deviates significantly from this established baseline is flagged as potentially malicious. This allows anomaly-based IDS to detect previously unseen or zero-day attacks, since it does not depend on prior knowledge of the specific attack, but it is more prone to false positives, because legitimate but unusual behavior (a new business process, a seasonal traffic spike) can also trigger alerts, and the baseline itself must be carefully trained and periodically updated to remain accurate as normal usage patterns evolve.

18. Apply IDS/IPS mechanisms to prevent a distributed denial-of-service (DDoS) attack.

Answer: In a DDoS scenario, an attacker uses a large number of compromised hosts (a botnet) to flood a target server with traffic, exhausting its bandwidth, connection capacity, or application resources. An IDS deployed at the network perimeter can detect this by identifying anomalous traffic patterns — a sudden, massive spike in connection requests from many distinct (often spoofed) source IPs targeting the same destination, unusually high SYN packet rates without

completed handshakes (indicating SYN flood behavior), or traffic that matches known DDoS tool signatures — and generate real-time alerts for the security team.

An IPS goes a step further by taking automated, inline action rather than just alerting: it can immediately rate-limit or drop traffic from source IPs exhibiting attack behavior, block traffic matching a recognized DDoS signature, enforce SYN cookies or connection-rate thresholds to mitigate SYN floods, and dynamically reconfigure firewall rules to blacklist offending sources in real time. For volumetric attacks that exceed local capacity, the IPS/IDS data is typically also used to trigger upstream mitigation, such as engaging an ISP or cloud-based DDoS scrubbing service to filter malicious traffic before it ever reaches the enterprise network. Combining anomaly-based detection (to catch novel attack patterns) with signature-based detection (to instantly recognize known DDoS tools) and automated inline blocking gives the most effective real-time defense against DDoS attacks.

19. Evaluate the differences between IDS and IPS in terms of response to detected threats.

Answer: An Intrusion Detection System (IDS) is fundamentally a passive, monitoring technology: it is typically deployed out-of-band (via a network tap or mirrored port), analyzing a copy of traffic. When it detects suspicious or malicious activity, it generates alerts, logs the event, and notifies security administrators — but it does not itself take direct action to stop the traffic. This means a real human or automated downstream system must review and respond to the alert, which introduces a response delay, though it also means the IDS cannot itself become a single point of failure or accidentally block legitimate traffic, since it never sits directly in the traffic path.

An Intrusion Prevention System (IPS), by contrast, is an active, in-line technology, meaning all traffic must physically pass through it. When it detects a threat matching its signatures or anomaly thresholds, it can automatically block, drop, or reject the malicious packets/connection in real time, without waiting for human intervention — effectively stopping the attack as it happens rather than just reporting it afterward. This makes IPS far more effective at preventing damage from fast-moving threats, but it also introduces risk: because it sits inline, a misconfigured IPS or a false positive can block legitimate traffic, causing a self-inflicted denial of service, and any IPS failure or performance bottleneck directly impacts network availability. In short, IDS trades speed of response for lower operational risk, while IPS trades some risk for immediate, automated protection.

20. Propose improvements to reduce false positives in an IDS while maintaining high detection accuracy.

Answer: Several complementary strategies can reduce false positives without sacrificing detection accuracy. First, adopt a hybrid detection approach that combines signature-based detection (for precise, low-false-positive matching of known threats) with anomaly-based detection (for catching novel attacks), and correlate alerts from both — treating an event flagged by multiple methods as higher-confidence than one flagged by a single method alone. Second, continuously tune the anomaly baseline using representative, up-to-date traffic data specific to the organization's actual usage patterns, rather than relying on generic thresholds, and retrain/update it periodically as legitimate usage patterns evolve (e.g., after adopting new applications or business processes).

Third, apply contextual and behavioral correlation — for example, using a SIEM to combine IDS alerts with contextual data such as asset criticality, user role, and time of day, so that an anomaly on a low-value test server during business hours is scored differently than the same anomaly on a critical production database server at 3 a.m. Fourth, incorporate machine learning models trained on labeled historical data (including known false positives) to progressively refine detection thresholds and reduce recurring false alarms. Fifth, implement a tiered alerting/whitelisting system where recurring, manually-verified benign patterns are explicitly whitelisted, and regularly review and prune outdated rules and signatures. Together, these measures reduce alert fatigue for security analysts while preserving — and often improving — the IDS's ability to catch genuine threats.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient